



Par Robert DIASSÉ

Création d'une Liste de Tâches (To-Do List)

Étape 1 : Structure de Base HTML et CSS

HTML

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>To-Do List</title>
  <style>
    /* style de votre choix */
    /* ou */
    /* style minimaliste proposé */
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
    }
    #todo-container {
      max-width: 400px;
      margin: auto;
    }
    #new-task-input {
      width: 80%;
      padding: 10px;
      margin-right: 10px;
    }
    #add-task-button {
      padding: 10px 20px;
    }
    .task {
      display: flex;
      justify-content: space-between;
      align-items: center;
      padding: 10px;
      border: 1px solid #ccc;
```

```
        margin-top: 10px;
    }
    .task.done {
        text-decoration: line-through;
        color: #888;
    }
    .task button {
        margin-left: 10px;
    }
</style>
</head>
<body>
    <div id="todo-container">
        <h1>Ma To-Do List</h1>
        <input type="text" id="new-task-input" placeholder="Nouvelle tâche...">
        <button id="add-task-button">Ajouter</button>
        <ul id="task-list"></ul>
    </div>

    <script>
        // Le code JavaScript viendra ici
    </script>
</body>
</html>
```

Explication

1. Structure HTML de base :

- **Balise <html>** : Définit le document HTML.
- **Balise <head>** : Contient les métadonnées du document et les styles CSS.
- **Balise <body>** : Contient le contenu visible de la page.

2. Styles CSS minimaliste :

- **body** : Style général pour le corps du document.
- **#todo-container** : Conteneur central pour la To-Do List.
- **#new-task-input, #add-task-button** : Styles pour l'input et le bouton d'ajout.
- **.task** : Style pour chaque tâche.
- **.task.done** : Style pour les tâches marquées comme terminées.

3. HTML pour l'interface :

- **Input** : Champ pour entrer une nouvelle tâche.
- **Button** : Bouton pour ajouter une tâche.
- **<ul id="task-list">** : Liste non ordonnée pour afficher les tâches.

Étape 2 : Ajouter une Nouvelle Tâche

JavaScript

```
document.addEventListener('DOMContentLoaded', () => {
  const taskInput = document.getElementById('new-task-input');
  const addTaskButton = document.getElementById('add-task-button');
  const taskList = document.getElementById('task-list');

  addTaskButton.addEventListener('click', () => {
    const taskText = taskInput.value.trim();
    if (taskText !== '') {
      const taskItem = document.createElement('li');
      taskItem.className = 'task';
      taskItem.innerHTML = `
        <span>${taskText}</span>
        <div>
          <button class="complete-button">Terminer</button>
          <button class="delete-button">Supprimer</button>
        </div>
      `;
      taskList.appendChild(taskItem);
      taskInput.value = '';
    }
  });
});
```

Explication

1. **document.addEventListener('DOMContentLoaded', ...)** : Assure que le DOM est complètement chargé avant d'exécuter le code JavaScript.
2. **Sélection des éléments** :
 - **taskInput** : Sélectionne l'input pour les nouvelles tâches.
 - **addTaskButton** : Sélectionne le bouton d'ajout.
 - **taskList** : Sélectionne la liste des tâches.
3. **Ajouter un événement 'click' au bouton** :
 - **addTaskButton.addEventListener('click', ...)** : Ajoute un écouteur d'événement au bouton.
 - **Vérification du texte de la tâche** : Trim l'input et vérifie qu'il n'est pas vide.
 - **Création de l'élément de tâche** : Crée un nouvel élément `<li>` avec le texte de la tâche et des boutons "Terminer" et "Supprimer".
 - **Ajout de la tâche à la liste** : Ajoute l'élément de tâche à la liste.
 - **Réinitialisation de l'input** : Vide le champ de texte après l'ajout de la tâche.

Une explication simple de **addEventListener** en JavaScript :

addEventListener

Description

`addEventListener` permet d'attacher un gestionnaire d'événements à un élément HTML. Cela signifie que lorsque l'événement spécifié se produit sur cet élément, une fonction de rappel (callback) est exécutée.

Syntaxe

```
element.addEventListener(event, function, useCapture(optionnel));
```

- **element** : L'élément HTML auquel vous souhaitez ajouter l'écouteur d'événement.
- **event** : Le type d'événement à écouter (par exemple, "click", "mouseover", "keydown").
- **function** : La fonction qui sera exécutée lorsque l'événement se produira.
- **useCapture** (optionnel) : Un booléen indiquant si l'événement doit être capturé au lieu d'être propagé (par défaut à false).

Exemple

HTML

```
<button id="monBouton">Cliquez-moi</button>
```

JavaScript

```
document.addEventListener('DOMContentLoaded', () => {  
  const bouton = document.getElementById('monBouton');  
  
  bouton.addEventListener('click', () => {  
    alert('Bouton cliqué !');  
  });  
});
```

Explication

1. **Chargement du DOM** : `document.addEventListener('DOMContentLoaded', ...)` garantit que le DOM est complètement chargé avant d'exécuter le code JavaScript.
2. **Sélection de l'élément** : `const bouton = document.getElementById('monBouton');` sélectionne le bouton avec l'ID `monBouton`.
3. **Attachement de l'écouteur d'événement** : `bouton.addEventListener('click', () => { alert('Bouton cliqué !'); });` attache un écouteur d'événement `click` au bouton, qui affiche une alerte lorsque le bouton est cliqué.

Points Clés

- **Multiples écouteurs** : Vous pouvez attacher plusieurs écouteurs au même élément pour le même type d'événement.
- **Suppression des écouteurs** : Vous pouvez supprimer un écouteur avec `removeEventListener`.

- **Compatibilité** : Compatible avec tous les navigateurs modernes.

Étape 3 : Marquer une Tâche comme Terminée

JavaScript

```
taskList.addEventListener('click', (event) => {  
  if (event.target.classList.contains('complete-button')) {  
    const taskItem = event.target.closest('.task');  
    taskItem.classList.toggle('done');  
  }  
});
```

Explication

1. Event delegation :

- `taskList.addEventListener('click', ...)` : Ajoute un écouteur d'événement 'click' à la liste des tâches (`taskList`) pour capturer les clics sur les boutons des tâches.

2. Vérification de la cible :

- `event.target.classList.contains('complete-button')` : Vérifie si l'élément cliqué est un bouton "Terminer".

3. Basculer la classe 'done' :

- `event.target.closest('.task')` : Trouve l'élément parent de la tâche.
- `taskItem.classList.toggle('done')` : Ajoute ou retire la classe 'done' pour marquer ou démarquer la tâche comme terminée.

Étape 4 : Supprimer une Tâche

JavaScript

```
taskList.addEventListener('click', (event) => {  
  if (event.target.classList.contains('delete-button')) {  
    const taskItem = event.target.closest('.task');  
    taskList.removeChild(taskItem);  
  }  
});
```

Explication

1. Vérification de la cible :

- `event.target.classList.contains('delete-button')` : Vérifie si l'élément cliqué est un bouton "Supprimer".

2. Suppression de la tâche :

- `event.target.closest('.task')` : Trouve l'élément parent de la tâche.
- `taskList.removeChild(taskItem)` : Supprime l'élément de tâche de la liste.

Code Complet

HTML et Javascript

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>To-Do List</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
    }
    #todo-container {
      max-width: 400px;
      margin: auto;
    }
    #new-task-input {
      width: 80%;
      padding: 10px;
      margin-right: 10px;
    }
    #add-task-button {
      padding: 10px 20px;
    }
    .task {
      display: flex;
      justify-content: space-between;
      align-items: center;
      padding: 10px;
      border: 1px solid #ccc;
      margin-top: 10px;
    }
    .task.done {
      text-decoration: line-through;
      color: #888;
    }
    .task button {
      margin-left: 10px;
    }
  </style>
</head>
<body>
  <div id="todo-container">
    <h1>Ma To-Do List</h1>
```

```

    <input type="text" id="new-task-input" placeholder="Nouvelle tâche...">
    <button id="add-task-button">Ajouter</button>
    <ul id="task-list"></ul>
  </div>

  <script>

    document.addEventListener('DOMContentLoaded', () => {
    const taskInput = document.getElementById('new-task-input');
    const addTaskButton = document.getElementById('add-task-button');
    const taskList = document.getElementById('task-list');

    addTaskButton.addEventListener('click', () => {
      const taskText = taskInput.value.trim();
      if (taskText !== '') {
        const taskItem = document.createElement('li');
        taskItem.className = 'task';
        taskItem.innerHTML = `
          <span>${taskText}</span>
          <div>
            <button class="complete-button">Terminer</button>
            <button class="delete-button">Supprimer</button>
          </div>
        `;
        taskList.appendChild(taskItem);
        taskInput.value = '';
      }
    });

    taskList.addEventListener('click', (event) => {
      if (event.target.classList.contains('complete-button')) {
        const taskItem = event.target.closest('.task');
        taskItem.classList.toggle('done');
      }

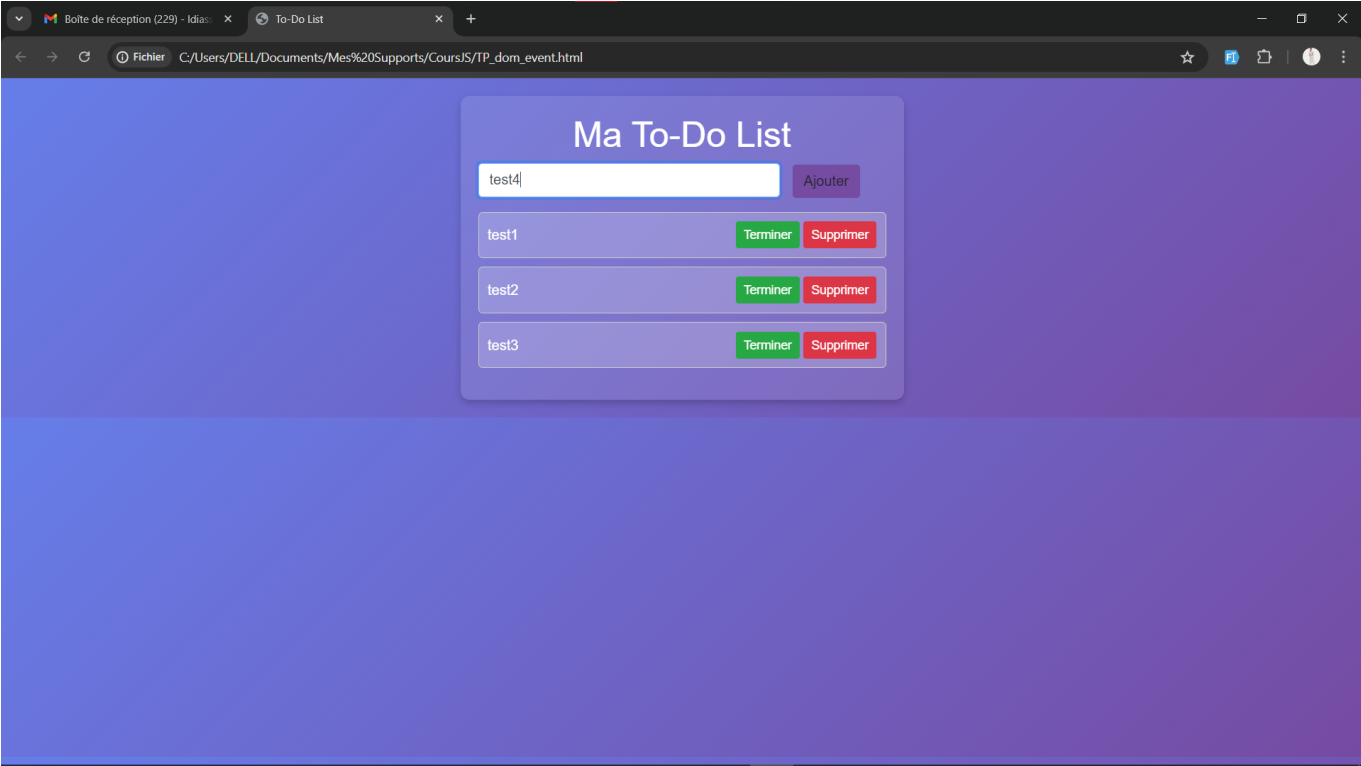
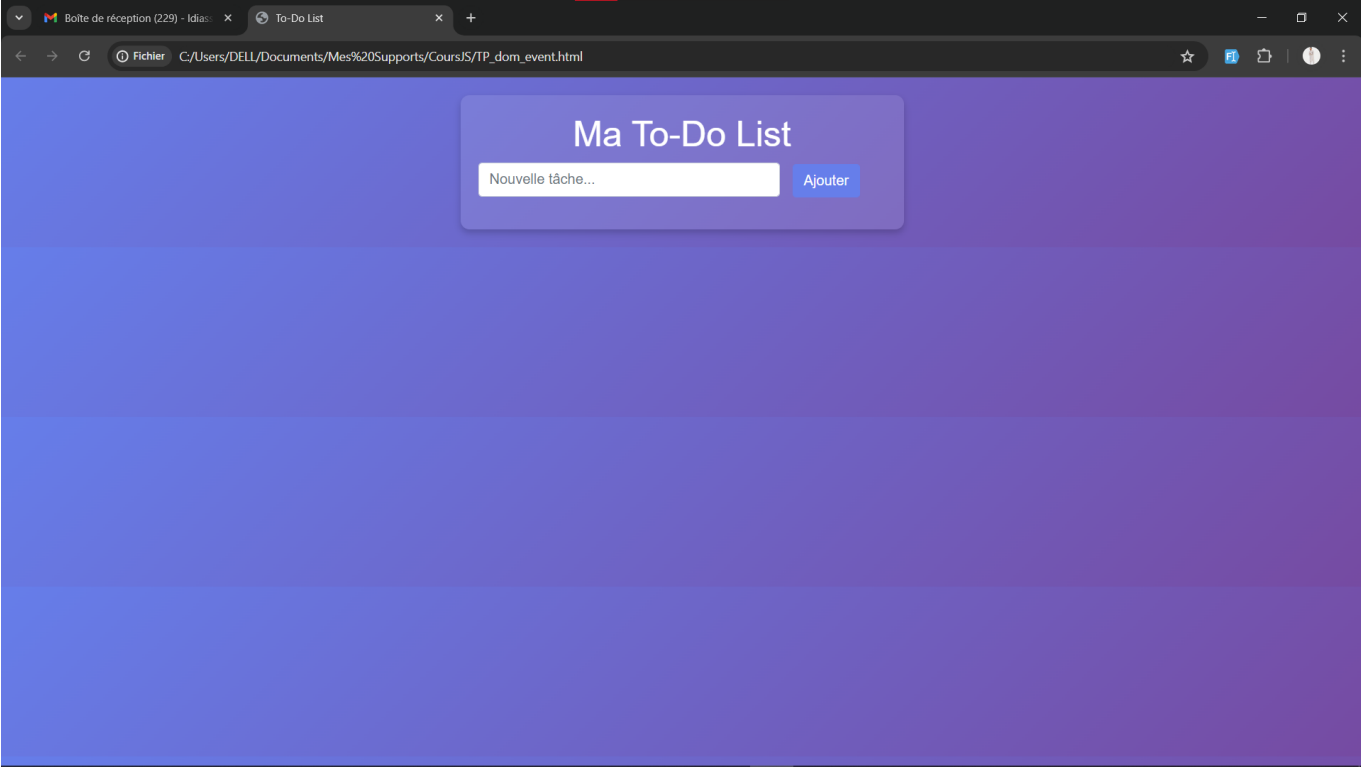
      if (event.target.classList.contains('delete-button')) {
        const taskItem = event.target.closest('.task');
        taskList.removeChild(taskItem);
      }
    });

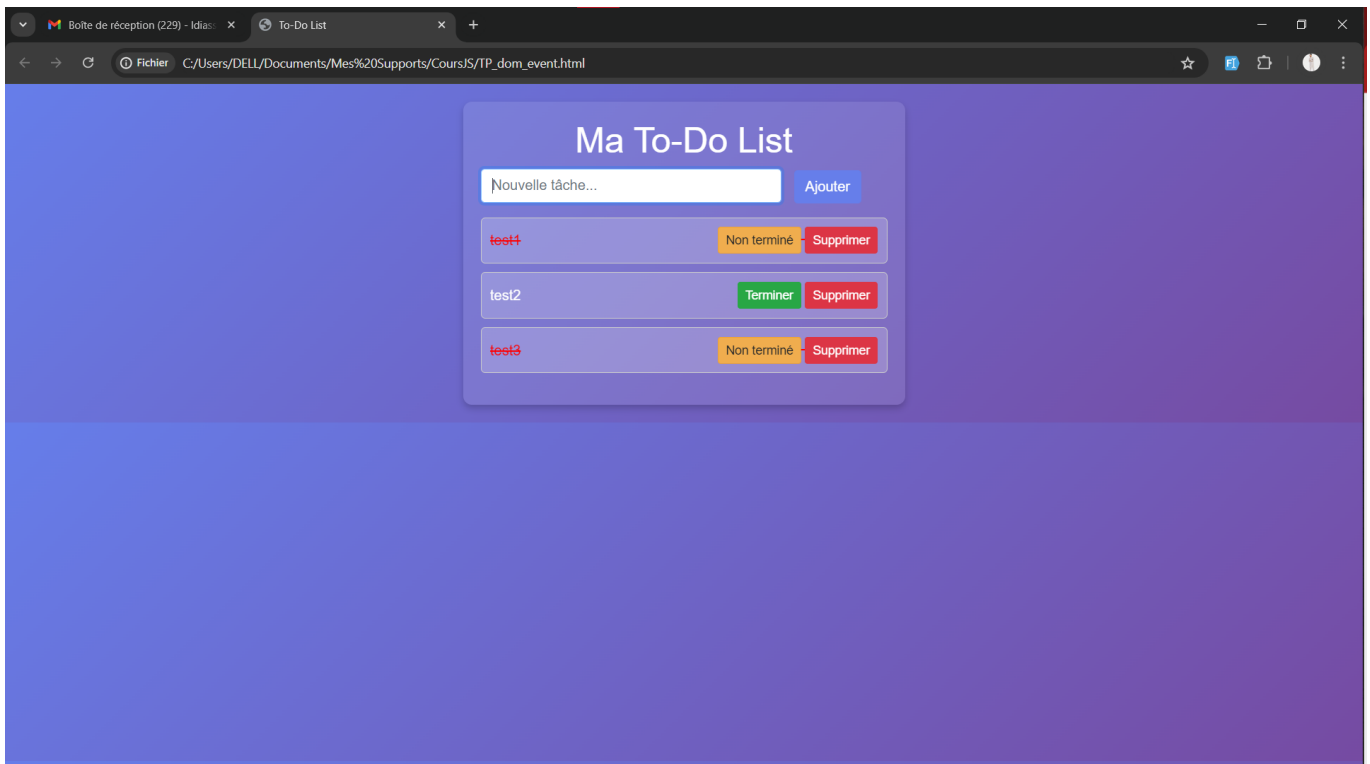
  });

  </script>
</body>
</html>

```

Prenez le temps de lire convenablement les explications et d'essayer le code étape par étape le rendu bien stylisé devrais pouvoir ressembler un peu à cela 🖱️🖱️🖱️🖱️:





Pour perfectionner cette To-Do List, vous pouvez ajouter plusieurs fonctionnalités supplémentaires pour enrichir l'expérience utilisateur et mieux gérer les éléments du DOM. Voici quelques perspectives de fonctionnalités que vous pourriez implémenter :

Fonctionnalités Additionnelles

1. Heure d'ajout :

- Afficher l'heure et la date où chaque tâche a été ajoutée.

2. Heure de suppression :

- Enregistrer et afficher l'heure de suppression de chaque tâche (peut être affiché dans une section d'historique).

3. Édition de tâches :

- Permettre aux utilisateurs d'éditer le texte des tâches existantes.

4. Priorité des tâches :

- Ajouter des niveaux de priorité (haute, moyenne, basse) et les afficher avec des couleurs différentes.

5. Catégories de tâches :

- Permettre la classification des tâches par catégorie (travail, personnel, etc.).

6. Notifications :

- Ajouter des notifications pour rappeler les tâches importantes.

7. Filtrage et recherche :

- Ajouter des options pour filtrer les tâches par état (terminé/non terminé) ou pour rechercher des tâches spécifiques.

FIN